

Sunt Văleanu Andrei de la 331CBb

Titlu: Implementarea protocolului CHORD in MPI

Descriere scurta:

Proiectul implementeaza o versiune statica a protocolului CHORD folosind MPI pentru comunicare intre procese. Fiecare nod cunoaste lista completa de noduri (inel static), construiesc finger table si proceseaza lookup-uri distribuite pentru gasirea unui key.

Ce am implementat:

`build_finger_table()`: calculez pentru fiecare i $start = (id + 2^i) \bmod RING_SIZE$ si folosesc `find_successor_simple()` pentru a gasi succesorul dintre nodurile existente.

`closest_preceding_finger()`: parcurg finger table de la cel mai mare la cel mai mic si aleg primul `finger.node` care se afla in intervalul $(self.id, key)$; daca nu gasesc, returnez succesorul.

`handle_lookup_request()`: adaug `self.id` in path, verific daca cheia e responsabilitatea mea (interval $(predecessor, self]$) sau a succesorului (interval $(self, successor]$) si trimit raspuns initiatorului; altfel forwardez catre `closest_preceding_finger`.

Initierea lookup-urilor locale: pentru fiecare cheie din fisierul `inX.txt`, se trimite un `TAG_LOOKUP_REQ` catre `self` pentru a porni rutarea.

Service loop distribuit: fiecare nod asculta MSG-uri `TAG_LOOKUP_REQ`, `TAG_LOOKUP_REP` si `TAG_DONE` si gestioneaza terminarea intregii retele.

Detalii si alegeri de implementare:

Variantei statice i se potriveste `find_successor_simple()`: caut succesorul in lista sortata de id-uri existente, nu pe intreg spatiul $0..RING_SIZE-1$.

Folosesc functia `in_interval()` pentru a trata corect intervalele circulare (wrap-around).

Maparea ID <-> MPI rank se realizeaza cu `all_ids[]` si `id_to_rank[]`; trimit/prim mesaje folosind rank-ul destinatiei.

Mesajele `LookupMsg` sunt trimise/recepute cu `MPI_BYTE` pentru a transmite structura ca octeti.

Am pus atentie la evitarea ciclurilor: `closest_preceding_finger` trebuie sa intoarca un nod diferit cand e posibil; in caz contrar rutarea poate esua.

Observatii si posibile imbunatatiri:

Adaugarea de mecanisme de timeout si retransmisie pentru stabilitate in retele instabile.

Verificare suplimentara pentru a preveni forward catre acelasi nod (self), eventual logging mai detaliat pentru debugging.

Extindere pentru join/leave dinamice si actualizarea finger table-urilor incremental.

Instructiuni de rulare:

Compile: `mpicc -o tema2 tema2.c`

Ruleaza: `mpirun -np <num_procs> ./tema2` (fiecare proces citeste fisierul `in<i>.txt`)

Testare: folositi setul de teste din directorul checker pentru verificari.

Multumesc.